



عملي

6

14

1680 sp

كلية الهندسة المعلوماتية

السنة الثالثة

التوارد Pipelining (2)

عملي مشترك

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 26/11/2024

بنيان الحواسيب 2

تحدثنا سابقاً عن التوارد في حالته المثالية أي الخالية من الأعطال، وسنتحدث في هذه المحاضرة عن التوارد بوجود أعطال.

أنواع الأعطال Hazards

أعطال التحكم

Control Hazards

ناتجة عن تعليمات القفز والتفرع
(j, beq, bne, ...)

أعطال المعطيات

Data Hazards

ناتجة عن ترابط المعطيات
البيني (طلب قراءة قيمة
سجل تم التعديل عليه
بتعليمة سابقة قبل تحديث
القيمة).

أعطال بنيوية

Structural Hazards

ناتجة عن التنازع على
الموارد العتادية resource
conflict وتداخل تعليمتان
على مورد واحد مثل (ذاكرة
- سجلات).

المراحل التي تمر بها التعليمات في المعالج المتوارد:

Fetch: جلب التعليمة.

1

Decode: تحديد التعليمة من أي نمط (J-type, I-type, R-type) ومعرفة معاملاتها (rs, rd, ...).

2

Execute: دخول المعاملات إلى ALU.

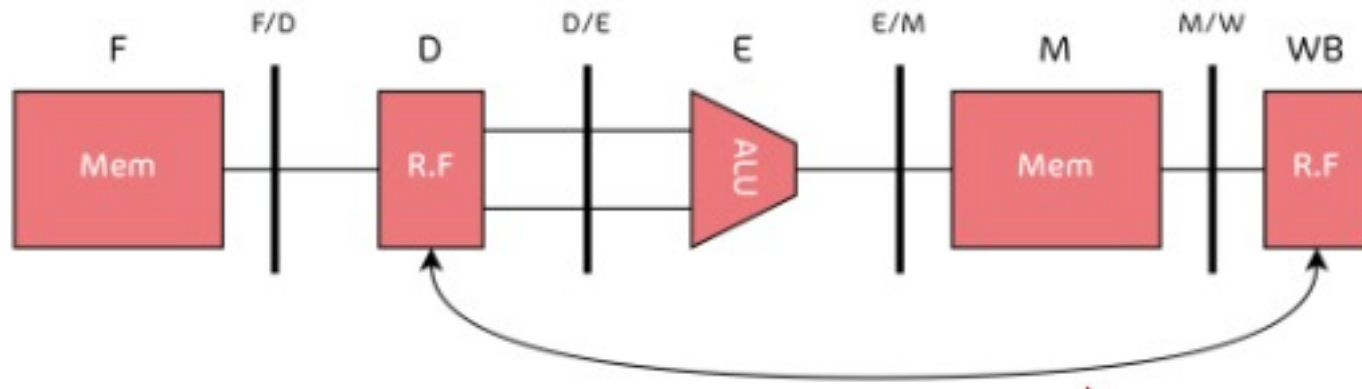
3

Memory: نفاذ إلى ذاكرة المعطيات بتعليمات (load, store).

4

Write Back: كتابة النتائج في ملف السجلات (بالعمليات الحسابية والمنطقية و load).

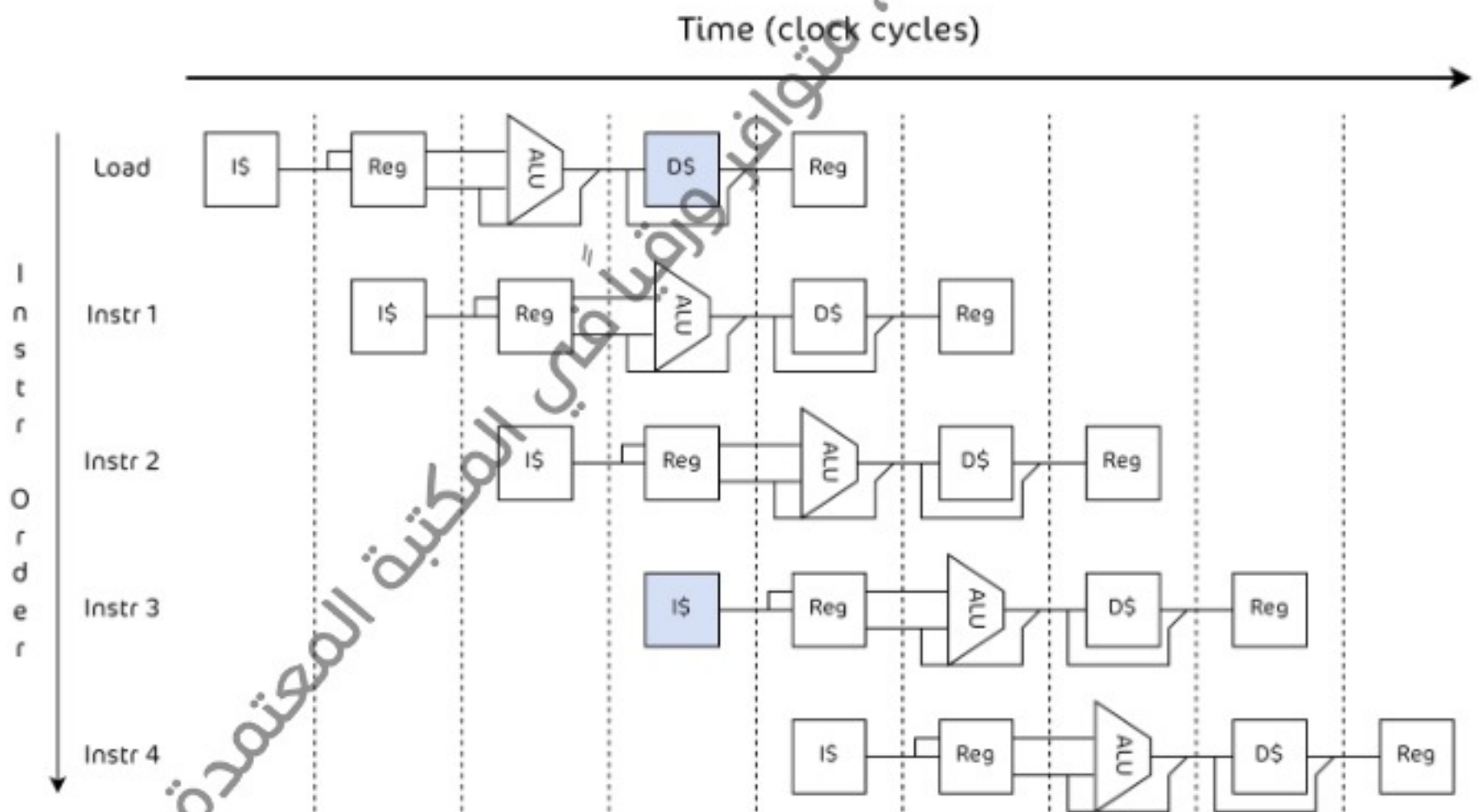
5



- هذا السهم يدل على أن R.F نفس ملف السجلات في مرحلتي D, WB.
- أما Memory يوجد 2 Memory إحداهما لل Data والأخرى للتعليمات.

الأعطال البنيوية Structural Hazards:

1. الذاكرة الوحيدة:



- نلاحظ في التعليم Load في الدور الرابع تكون بمرحلة Memory (مرحلة قراءة Data من الذاكرة) وفي الدور ذاته تكون التعليم Instr 3 بمرحلة Fetch (مرحلة قراءة تعليم من الذاكرة) وكلا هاتين المرحلتين تحدثان على نفس الذاكرة (بشرط امتلاك ذاكرة واحدة)، لكن قراءة الذاكرة ذاتها مرتين في الدورة الواحدة أمر غير ممكن!

الحلول:

- ذاكرة خابية للتعليمات وأخرى للمعطيات.
سليبيته: التكلفة العالية.

- التأخير الزمني Bubble, stall.
سليبيته: التأثير على أداء البرنامج بسبب الانتظار.

فكان الحل الأفضل:



تقسيم الذاكرة إلى 2 cache إحداهما للـ Data والأخرى للـ Instruction.

ملاحظة:

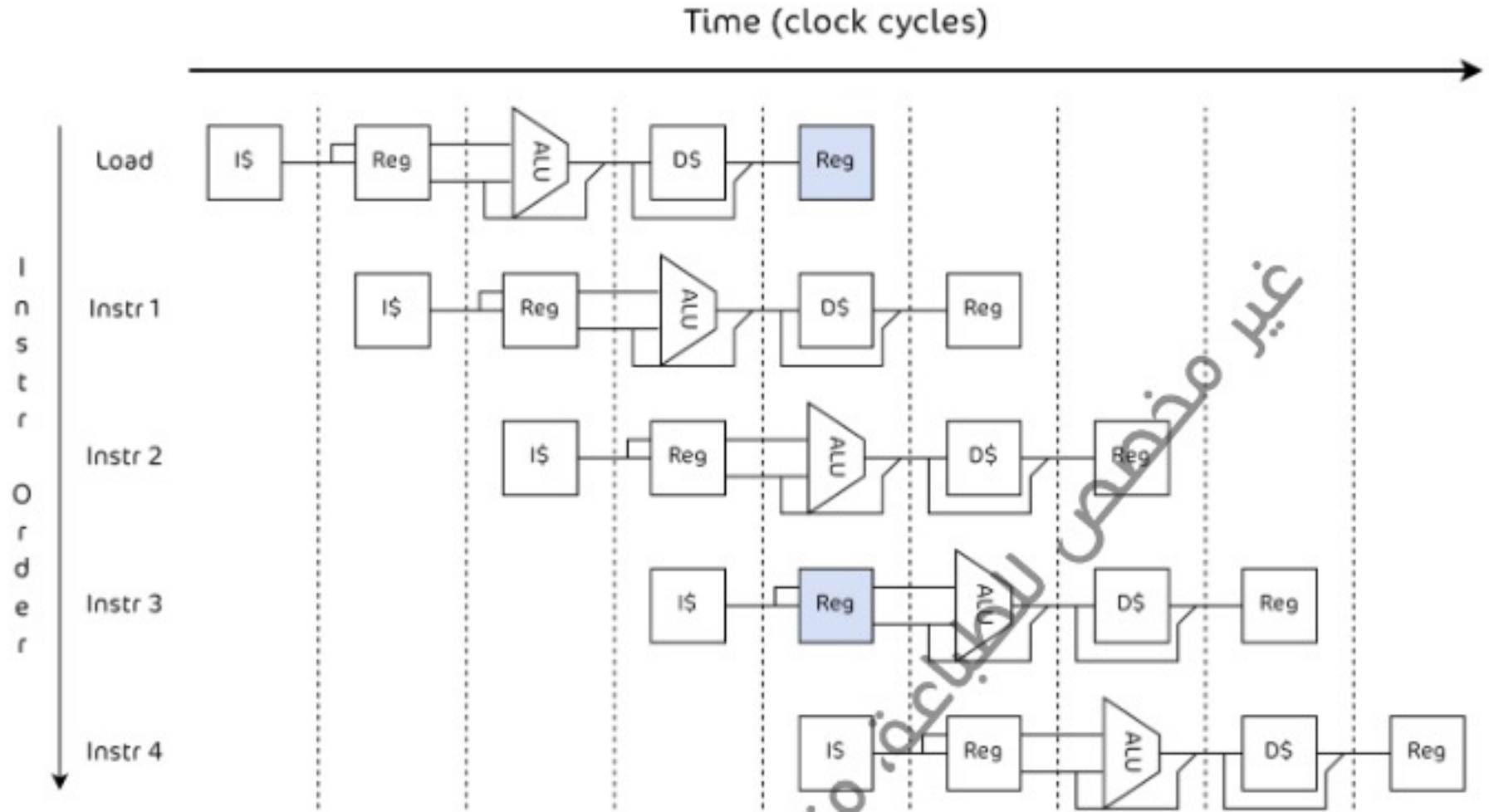
لدينا التعليمات التالية:

load	F	D	E	M	W		
addi		F	D	E	M	W	
add			F	D	E	M	W
or				F			

نلاحظ أن Memory في تعليمة load و Fetch في تعليمة or هي الدور ذاته لذا نقوم بتأخير Fetch فتصبح في الدور الخامس:

load	F	D	E	M	W		
addi		F	D	E	M	W	
add			F	D	E	M	W
or				-	F		

- لكن في هذا التأخير أصبح لدينا تنازع مرة أخرى على الذاكرة نفسها، فهل تعتبر المشكلة محلولة أم لا؟ في الواقع المشكلة قد حُلَّت وذلك لأنه كما نعلم في Pipeline التعليمات جميعها ستمر بالمرحلة الخمسة سواء تم تنفيذ هذه المراحل أم لا، والتعليمة add كما نعلم لا تنفذ للذاكرة (أي لا تنفذ مرحلة Mem) لأن add تعليمة حسابية.



- نلاحظ في التعليمة load السابقة في الدور الخامس بمرحلة Write Back وفي التعليمة Instr 3 بنفس الدور مرحلة Decode (قراءة ما تحتويه معاملات السجلات)، بالتالي أصبح هناك تنافس على ملف السجلات.
- بداية نعلم أن زمن النفاذ للسجلات أسرع من زمن باقي المكونات، وبمثالنا بالمحاضرة السابقة كانت ازمة التأخير لكل من:

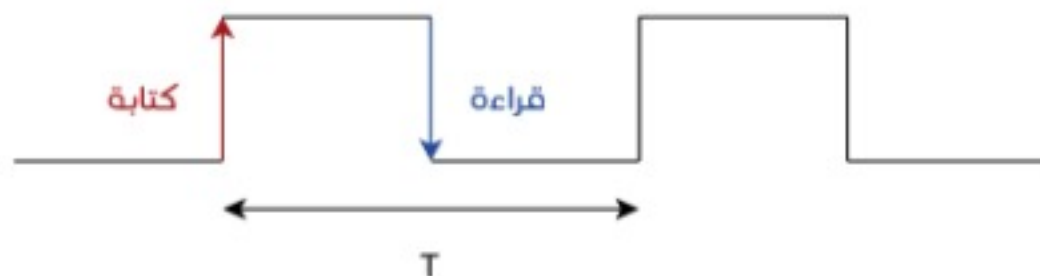
$$\text{Mem} = 2$$

$$\text{ALU} = 1.2$$

$$\text{R.F} = 0.8$$

إذاً $T = 2$ وبما أن $\text{R.F} = 0.8$ بالتالي أصبح يمكننا بالدور الواحد النفاذ إلى ملف السجلات أكثر من مرتين فكان الحل:

الكتابة في السجل أثناء النصف الأول لدورة الساعة والقراءة من السجل في النصف الثاني، بالتالي يمكن القراءة والكتابة في السجل في دور الساعة نفسه.



يسمى هذا الحل بالنفاذ المزدوج.

أعطال المعطيات Data Hazards:

تحدث بسبب اعتماد تعليمة على نتيجة تعليمة سابقة (لا تزال موجودة في قناة التوارد PL)

الحلول الممكنة:

- التأخير الزمني Stalls / إضافة فقاعات Bubbles (حل عتادي يحتاج إضافة دارة كشف الأعطال Hazard Detection Unit).
 - إضافة دارة إحالة Bypassing / Forwarding Unit للحصول على المعطيات دون انتظار الكتابة النهائية على الوجهة (حل عتادي يحتاج أيضاً إلى دارة كشف الأعطال Hazard Detection Unit).
 - إعادة ترتيب البرنامج عن طريق المترجم Compiler بحيث يتم المباشرة بين التعليمات المترابطة، وإضافة NOP عند الحاجة (حل برمجي):
- In MIPS: NOP = "no operation" = 00...0 (32 bits)
= sll \$0, \$0, 0
#sll: shift left logical

مثال

بفرض أن محتوى السجل الابتدائي لأي سجل هو رقم السجل * 2 (مثلاً محتوى \$3 هو 6)، ماهو المحتوى النهائي للسجلات في نهاية تنفيذ كل تعليمة من التعليمات التالية على معالج متعدد الأدوار MC وعلى معالج متوارد PL؟

```
addi $9, $3, 1
```

```
sub $5, $6, $9
```

1. على معالج متعدد الأدوار MC:

```
addi $9, $3, 1 # $9 = 6 + 1 = 7
```

```
sub $5, $6, $9 # $5 = 12 - 7 = 5
```

طريقة تنفيذ التعليمتين:

F D E W F D E W

لا يوجد مرحلة M في MC بالتعليمات الحسابية.

2. على معالج متوارد PL:

```
addi $9, $3, 1 # $9 = 6 + 1 = 7
```

يتم حفظ الناتج 7 في R.F بالدور 5 (مرحلة W)

```
sub $5, $6, $9
```

\$5 = ??

ترابط معطيات



طريقة التنفيذ:

1	F	D	E	M	W
2		F	D	E	

هنا السؤال ما هي القيم على مداخل ALU في مرحلة E؟

إذا كانت قيمة \$9 هي 18 فتتخذ البرنامج يكون خاطئاً، وإذا كانت القيمة 7 فهي لم تحفظ بعد، حيث يكون حفظها في نهاية الدور الخامس.

$$\$6 = 12$$

$$\$9 = ??$$

ALU

7?

18?

تمرين

بفرض لدينا البرنامج التالي ينفذ على معالج MIPS متوارد، حدد ترابطات المعطيات الموجودة في البرنامج، ثم مثل أدوار التنفيذ بفرض استخدام طريقة التأخير الزمني Stall (إضافة فقاعات Bubbles) لمعالجة أعطال المعطيات بفرض عدم وجود أعطال بنيوية، وبفرض إمكانية النفاذ المزدوج لملف السجلات (إمكانية الكتابة والقراءة بنفس الدور).

(قراءة السجلات تتم في المرحلة D)

■ في الذاكرة عند ذكر معالج MIPS متوارد $\Leftarrow$ الذواكر لا يوجد أعطال بنيوية عليها و R.F له إمكانية النفاذ المزدوج.

تحدث أعطال المعطيات عندما يكون لدى تعليمة سابقة تقوم بالكتابة على سجل وتعليمة لاحقة تقوم بالقراءة من نفس هذا السجل (Read after write).



في الشام تتغير جغرافية جسدي

تصبح كريات دمي خضراء

وأبجديتي خضراء



```
and    $t1,    $t3,    $t2
ori     $t0,    $t3,    -1
sub     $t5,    $a0,    $t0
and     $t4,    $t2,    $t6
lw      $t2,    0($t5)
sw      $t4,    4($t2)
or      $t5,    $t6,    $t2
```

بداية في التعليمة الأولى نقوم بالتركيز على السجل \$t1 الذي تتم الكتابة عليه، هل لدينا تعليمة أخرى تحاول القراءة منه؟ لا يوجد. بالتالي لا يوجد ترابط معطيات.

```
and    $t1,    $t3,    $t2
ori     $t0,    $t3,    -1
sub     $t5,    $a0,    $t0
and     $t4,    $t2,    $t6
lw      $t2,    0($t5)
sw      $t4,    4($t2)
or      $t5,    $t6,    $t2
```

السجل \$t0 تتم الكتابة عليه في ori ونجد أنه تتم القراءة منه في التعليمة sub، إذاً يوجد ترابط معطيات وذلك خوفاً من وصول مرحلة قراءته قبل أن يتم التعديل عليه في مرحلة lw.

```
and    $t1,    $t3,    $t2
ori     $t0,    $t3,    -1
sub     $t5,    $a0,    $t0
and     $t4,    $t2,    $t6
lw      $t2,    0($t5)
sw      $t4,    4($t2)
or      $t5,    $t6,    $t2
```

في التعليمة sub لدينا السجل \$t5 تتم الكتابة عليه ولدينا في التعليمة lw قراءة من نفس السجل، بالتالي يوجد ترابط معطيات.

```
and    $t1,    $t3,    $t2
ori     $t0,    $t3,    -1
sub     $t5,    $a0,    $t0
and     $t4,    $t2,    $t6
lw      $t2,    0($t5)
sw      $t4,    4($t2)
or      $t5,    $t6,    $t2
```

في التعليمة and لدينا السجل \$t4 تتم الكتابة عليه، ونلاحظ في التعليمة sw (من أكثر التعليمات التي ينسى الطلاب طريقة عملها)

$$sw \text{ rt, lm (rs)}$$

$$\text{address} = \text{lm} + \text{rs}$$
 بعد حساب ال address نقوم بتخزين محتوى السجل rt به، بالتالي في التعليمة sw نقوم بقراءة \$t2 وجمعه مع 4 لتحديد العنوان ونقوم بقراءة \$t4 لتخزين محتواه في العنوان الذي قمنا بحسابه. إذاً يوجد ترابط معطيات بين and و sw.

```

and    $t1,    $t3,    $t2
ori     $t0,    $t3,    -1
sub     $t5,    $a0,    $t0
and     $t4,    $t2,    $t6
lw      $t2,    0($t5)
sw      $t4,    4($t2)
or      $t5,    $t6,    $t2

```

السجل \$t2 تتم الكتابة عليه في التعليمة lw والقراءة منه في or و sw.

تمثيل أدوار التنفيذ بفرض استخدام طريقة التأخير الزمني stall (الفقاعات Bubble) لمعالجة أعطال المعطيات:

■ ملاحظة: أشكال التأخير الزمني:

- (1) 
- (2) 
- (3) 

ويفضل الشكل (1) وهو شكل فقاعة.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
F	D	E	M	W											
	F	D	E	M	W										
		F			D	E	M	W							
					F	D	E	M	W						
						F		D	E	M	W				
							F			D	E	M	W		
									F	D	E	M	W		
										F	D	E	M	W	

■ ملاحظة: أمكن هنا إجراء W و D على نفس السجل في نفس الدور حسب خاصية النفاذ المزدوج إلى ملف السجلات.

$$CPI = \frac{\{5 + (IC - 1) + stalls\}}{IC} = \frac{5 + 6 + 5}{7} = \frac{16}{7} = 2.3$$

stalls هي عدد الادوار الضائعة الممثلة بعدد الاعمدة الحاوية على فقاعات.

في منهاجنا هذه السنة تتم قراءة من ملف السجلات في مرحلة Decode فقط وليس Execute كما كانت في السنوات السابقة.

تمرين

حدد ترابطات المعطيات الموجودة في البرنامج، ثم مثل أدوار التنفيذ باستخدام طريقة التأخير الزمني stall لمعالجة الأعطال.

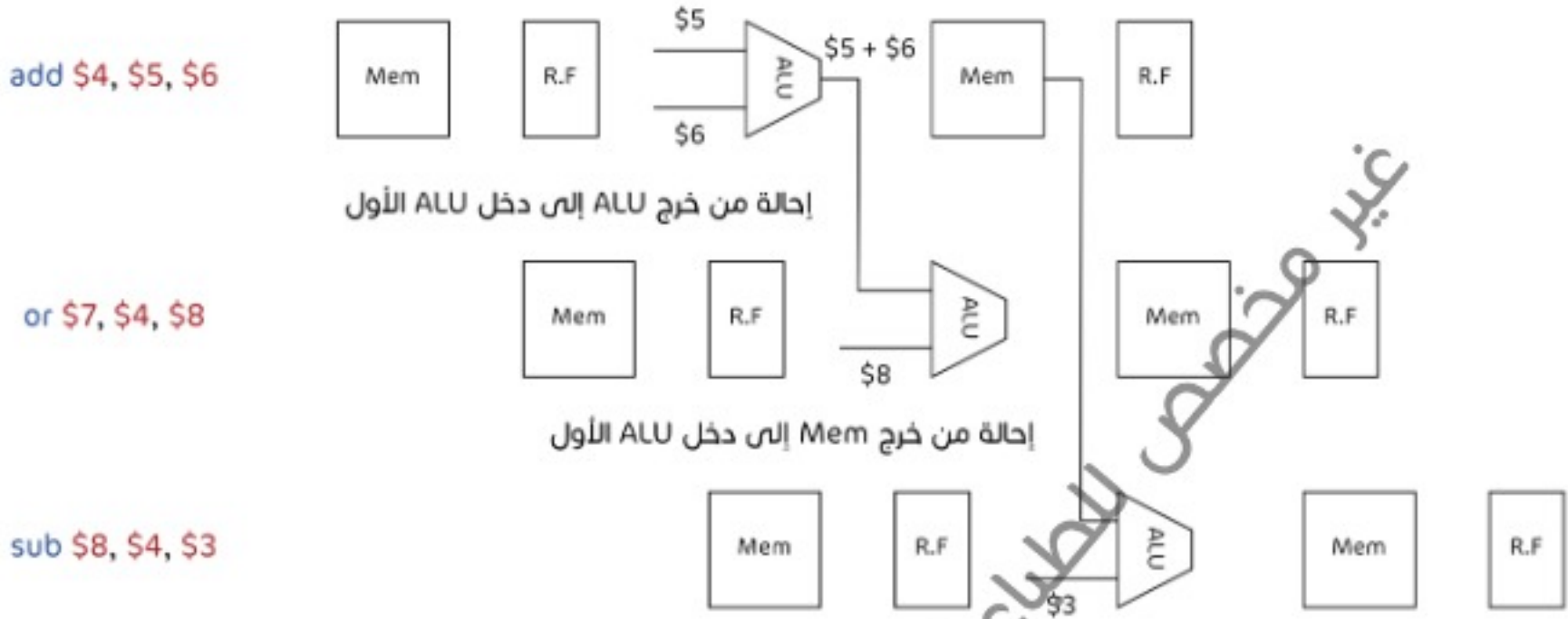


1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
F	D	E	M	W											
	F			D	E	M	W								
				F	D	E	M	W							
					F	D	E	M	W						
						F	D	E	M	W					
							F	D	E	M	W				
								F	D	E	M	W			
									F	D	E	M	W		
										F	D	E	M	W	
											F	D	E	M	W

$$CPI = \frac{\{5 + (IC - 1) + stalls\}}{IC} = \frac{5 + 6 + 5}{7} = \frac{16}{7} = 2.3$$

أعد الدل بفرض وجود دائرة إحالة Forwarding Unit، مثل أدوار التنفيذ وحدد مكان حدوث الإحالات، ثم احسب CPI.

■ مثال توضيحي لدائرة الإحالة:



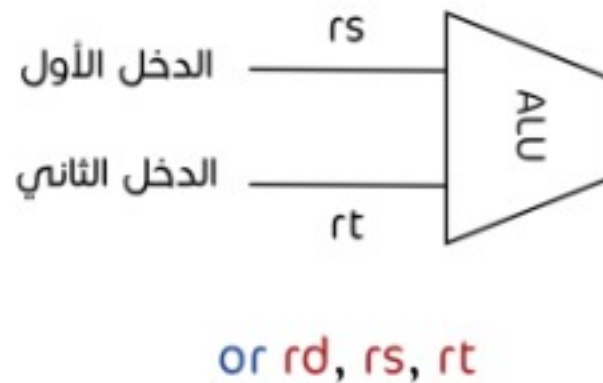
■ الإحالة تتم إما من خرج ALU أو من خرج Mem، والإحالة تكون عبارة عن خط شاقولي وليس مائل.

لنعود إلى التمرين:

1	2	3	4	5	6	7	8	9	10	11	12	13
F	D	E	M	W								
	F	D	E	M	W							
		F	D	E	M	W						
			F	D	E	M	W					
				F	D	E	M	W				
					F	D	E	M	W			
						F	D	E	M	W		

تسمى إحالة
السجل \$9 من
خرج ALU إلى
دخل ALU الثاني

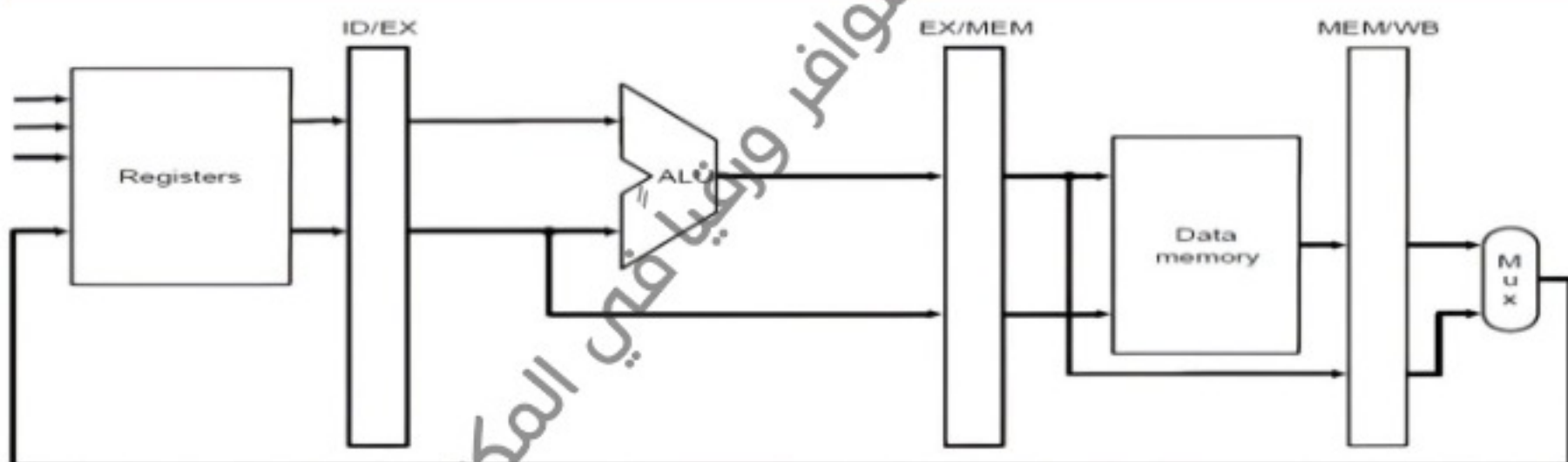
تسمى إحالة
السجل \$6 من
خرج MEM إلى
دخل ALU الأول



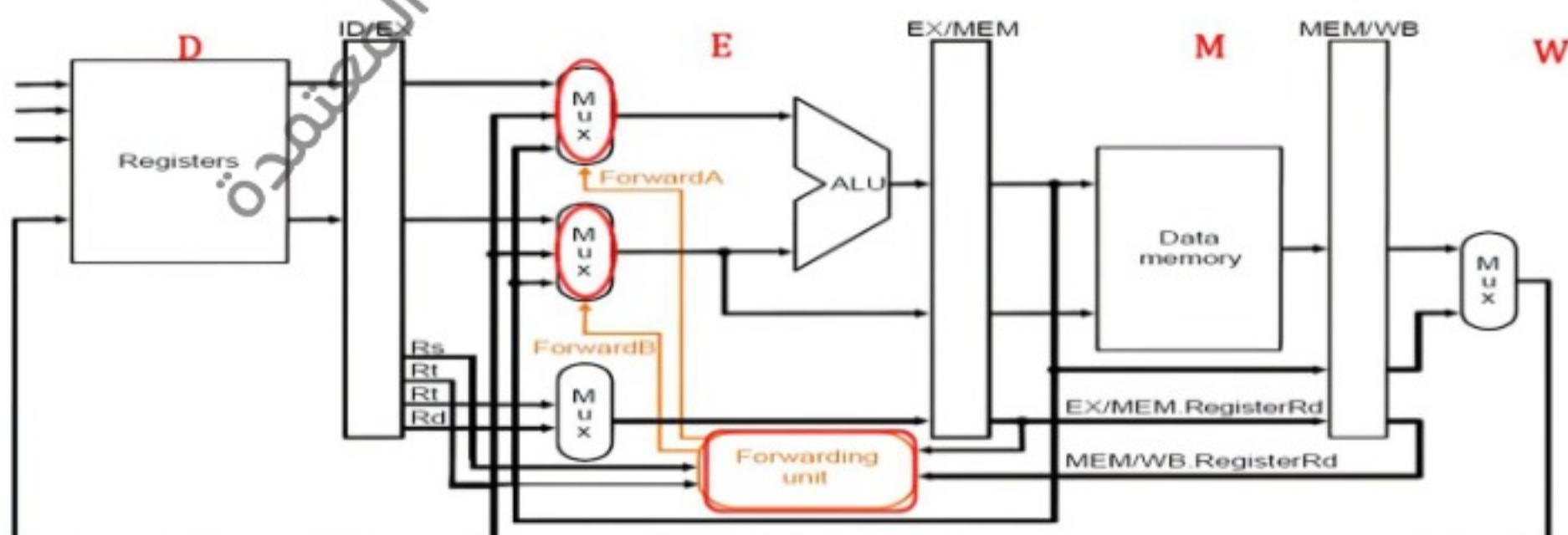
$$CPI = \frac{\{5 + (IC - 1) + stalls\}}{IC} = \frac{5 + 6 + 0}{7} = \frac{11}{7} = 1.57$$

حل أعطال المعطيات باستخدام دائرة الإحالة Forwarding Unit

1. No Forwarding



2. With Forwarding



تمرین

بفرض لدينا البرنامج التالي ينفذ على معالج MIPS متوارد، حدد ترابطات المعطيات الموجودة في البرنامج، ثم مثل أدوار التنفيذ بفرض وجود دائرة إحالة.

1 and \$8 \$9 \$4
2 ori \$5 \$3 5
3 add \$6 \$4 \$8
4 or \$2 \$6 \$7
5 lw \$9 0(\$3)
6 sw \$4 4(\$2)
7 sub \$5 \$4 \$3

1	2	3	4	5	6	7	8	9	10	11
F	D	E	M	W						
	F	D	E	M	W					
		F	D	E	M	W				
			F	D	E	M	W			
				F	D	E	M	W		
					F	D	E	M	W	
						F	D	E	M	W

$$CPI = \frac{\{5 + (IC - 1) + stalls\}}{IC} = \frac{4 + 6 + 0}{7} = \frac{11}{7} = 1.57$$

وظيفة

حدد ترابطات المعطيات ثم مثل أدوار تنفيذ البرنامج التالي على المعالج المتوارد بخمس مراحل، واحسب CPI (بفرض عدم وجود أعطال بنيوية) وذلك بفرض معالجة أعطال المعطيات:

1- باستخدام طريقة التأخير الزمني stalls.

2- باستخدام دائرة الإحالة F.U.

```

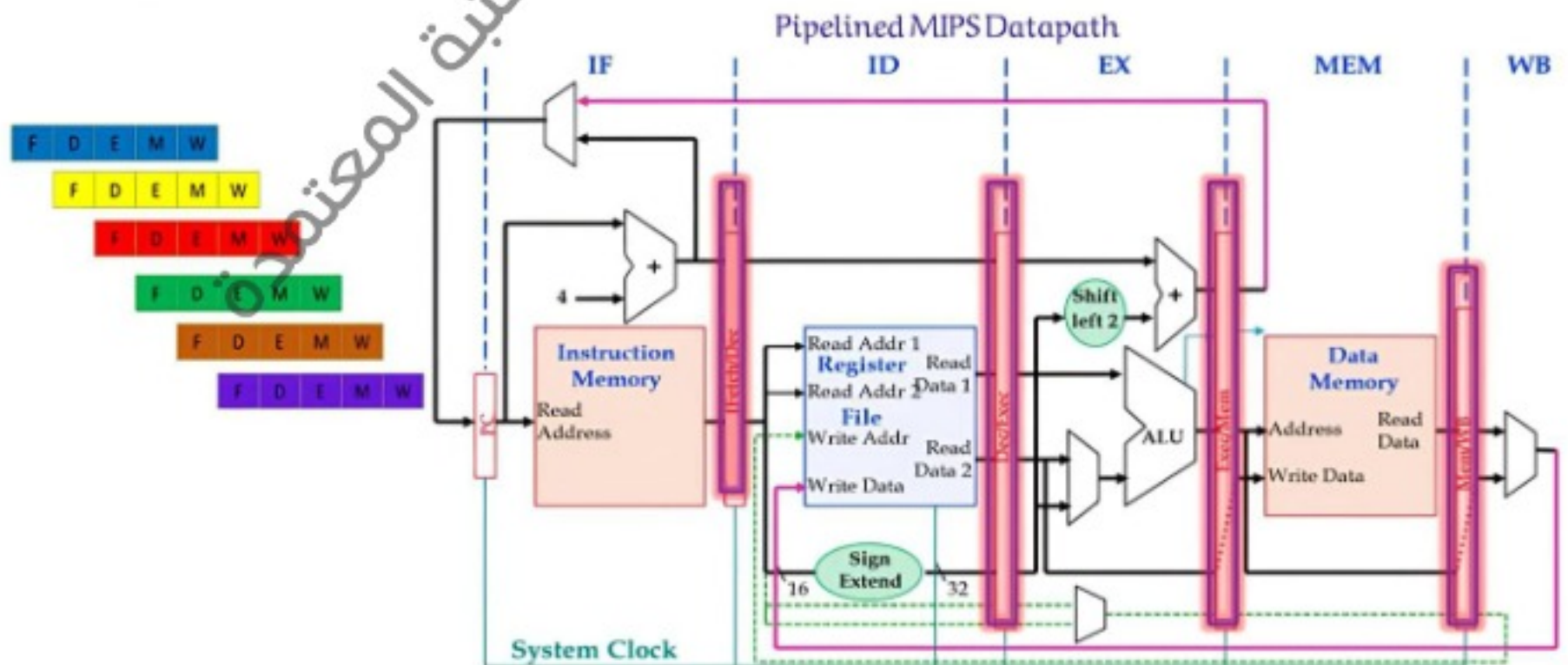
1  or   $t5  $t3  $t2
2  add  $t0  $t3  $t5
3  addi $t3  $t0  2
4  and  $t4  $t0  $t1
5  lw   $t2  0($t3)
6  sub  $t5  $t4  $t3
7  sw   $t5  4($t8)
  
```

حل وظيفة المحاضرة السابقة

بفرض تم تنفيذ 6 تعليمات هي:

add → sub → and → sw → addi → lw

على المعالج المتوارد التالي وبفرض عدم وجود أية أعطال، حدد توزع التعليمات على المخطط المرفق في الدور السادس من بدء التنفيذ، ثم حدد في أي دور ساعة يتم حفظ ناتج التعليمات lw, addi, sub في ملف السجلات؟

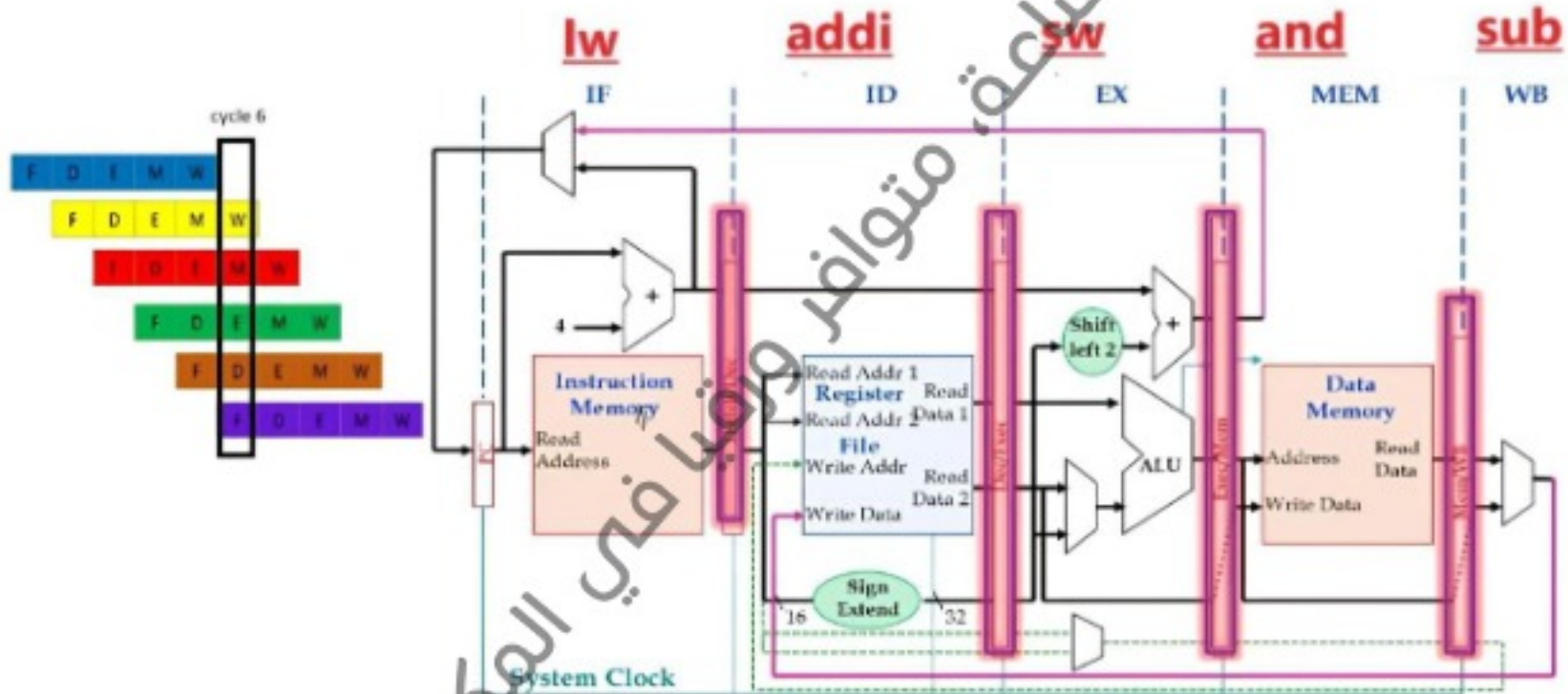


في الدور السادس

- تكون في مرحلة الـ Fetch تعليمة lw.
- تكون في مرحلة الـ Decode تعليمة addi.
- تكون في مرحلة الـ Execution تعليمة sw.
- تكون في مرحلة الـ Memory تعليمة and.
- تكون في مرحلة الـ Write Back تعليمة sub.

يتم حفظ ناتج كل من تعليمة:

- sub: يتم حفظ الناتج في السجلات في الدور السادس، لأنه يتم الكتابة على السجلات في المرحلة الخامسة (write back)، وحسب مخطط التوارد فإن هذه المرحلة لتعليمة (sub) هي في الدور السادس.
- addi: يتم حفظ الناتج في المسجلات في الدور التاسع.
- lw: يتم حفظ الناتج في السجلات في الدور العاشر.



The END

See you soon